

✓ DATA ENGINEERING LABORATORY

Experiment No. 1

Student Details

Name: Rohit Swami

Roll No.: 124A8113

Subject: Data Engineering

Title

Exploratory Data Analysis (EDA) and Data Cleaning using Python

Aim

To perform Exploratory Data Analysis (EDA) on a CSV dataset using Python and Pandas, identify missing values, duplicate records, inconsistent data types, detect outliers, clean the dataset, generate statistical summaries, visualize the data, and prepare a clean dataset for further analysis.

Objectives

- Load a CSV dataset into a Pandas DataFrame.
 - Explore the structure of the dataset.
 - Detect missing values and duplicate records.
 - Identify incorrect data types.
 - Perform data cleaning.
 - Detect statistical outliers.
 - Generate descriptive statistics.
 - Create meaningful visualizations.
 - Save the cleaned dataset.
-

Theory

Exploratory Data Analysis (EDA) is the process of examining and understanding a dataset before performing advanced analysis or building machine learning models. It

helps identify data quality issues, discover patterns, understand relationships among variables, and summarize the main characteristics of the dataset.

Real-world datasets often contain missing values, duplicate records, inconsistent data types, invalid entries, and outliers. Data cleaning improves the quality of the dataset by correcting or removing such issues, making the data reliable for analysis.

Python libraries such as **Pandas**, **NumPy**, **Matplotlib**, and **Seaborn** provide powerful tools for data manipulation, statistical analysis, and visualization. Pandas is used for loading and processing data, while Matplotlib and Seaborn are used to create graphs such as histograms, boxplots, and heatmaps.

EDA is an important step in the ETL (Extract, Transform, Load) process followed in Data Engineering because it ensures that only high-quality data is used for downstream analytics and machine learning applications.

Software Requirements

- Python 3.11 or above
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn

Hardware Requirements

- Intel i3/i5/i7 Processor or equivalent
- Minimum 8 GB RAM
- 10 GB Free Disk Space

Procedure

1. Import the required Python libraries.
2. Load the CSV dataset into a Pandas DataFrame.
3. Display the first and last few records of the dataset.
4. Check the dimensions of the dataset.
5. Display column names.
6. Verify the data types of all columns.
7. Display dataset information.
8. Generate descriptive statistics.
9. Identify missing values.

10. Detect duplicate records.
 11. Handle missing values using suitable methods.
 12. Convert columns to appropriate data types.
 13. Detect outliers using the IQR method and boxplots.
 14. Generate histograms and other visualizations.
 15. Create a correlation heatmap.
 16. Save the cleaned dataset.
-

Algorithm

1. Start
 2. Import required libraries
 3. Load the dataset
 4. Explore the dataset
 5. Check missing values
 6. Check duplicate records
 7. Clean the dataset
 8. Convert data types
 9. Detect outliers
 10. Generate statistical summaries
 11. Visualize the dataset
 12. Save the cleaned dataset
 13. Stop
-

Flowchart

```
graph TD
    Start([Start]) --> ImportLibraries[Import Libraries]
    ImportLibraries --> LoadDataset[Load Dataset]
    LoadDataset --> ExploreDataset[Explore Dataset]
    ExploreDataset --> CheckMissingValues[Check Missing Values]
    CheckMissingValues --> CheckDuplicateRecords[Check Duplicate Records]
    CheckDuplicateRecords --> CleanDataset[Clean Dataset]
    CleanDataset --> ConvertDataTypes[Convert Data Types]
```

```
Detect Outliers
|
Generate Statistics
|
Create Visualizations
|
Save Clean Dataset
|
End
```

Observation

- Dataset loaded successfully.
- Missing values identified and handled.
- Duplicate records checked.
- Data types verified.
- Outliers identified using the IQR method and boxplot.
- Statistical summaries generated.
- Visualizations created successfully.
- Cleaned dataset prepared for further analysis.

Result

The dataset was successfully analyzed using Exploratory Data Analysis (EDA). Missing values, duplicate records, data types, and outliers were identified and handled appropriately. Statistical summaries and visualizations were generated, and the cleaned dataset was prepared for further analytics.

Conclusion

Exploratory Data Analysis is a fundamental step in Data Engineering that helps improve data quality before analysis. Using Python, Pandas, Matplotlib, and Seaborn, the dataset was successfully explored, cleaned, and visualized, making it suitable for further analytical and machine learning tasks.

Viva Questions

1. What is Exploratory Data Analysis (EDA)?
2. Why is EDA important?
3. What is data cleaning?
4. What are missing values?

5. What are duplicate records?
6. What are outliers?
7. What is the IQR method?
8. What is the purpose of the `describe()` function?
9. What is the use of the `info()` function?
10. What is ETL?
11. What is the difference between ETL and ELT?
12. What is correlation?
13. Why are heatmaps used?
14. What is the purpose of a boxplot?
15. Which Python libraries are commonly used for EDA?

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("scooter (1).csv")
```

```
df.head()
```

	month	trip_id	region_id	vehicle_id	started_at	ended_at	DURATION
0	May	1613335	202	9424537	5/21/2019 18:33	5/21/2019 18:40	0:07:03
1	May	1613639	202	9424537	5/21/2019 19:07	5/21/2019 19:12	0:04:57
2	May	1613708	202	9424537	5/21/2019 19:13	5/21/2019 19:15	0:01:14
3	May	1613867	202	9424537	5/21/2019 19:29	5/21/2019 19:36	0:06:58

```
print(type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
print(df.shape)
```

```
(1, 1)
```

```
print(df.head().to_string())
```

For more on scraping GitHub and how it may affect your rights please

```
df.shape
```

```
(34226, 11)
```

```
df.columns
```

```
Index(['month', 'trip_id', 'region_id', 'vehicle_id', 'started_at',
      'ended_at',
      'DURATION', 'start_location_name', 'end_location_name',
      'user_id',
      'trip_ledger_id'],
      dtype='object')
```

```
df.dtypes
```

```

      0
month      object
trip_id    int64
region_id  int64
vehicle_id int64
started_at object
ended_at   object
DURATION   object
start_location_name object
end_location_name object
user_id    int64
trip_ledger_id int64

```

```
dtype: object
```

```
df.describe()
df.describe(include='object')
```

	month	started_at	ended_at	DURATION	start_location_name	end_
count	34226	34226	34226	31918	34220	
unique	3	19354	18654	4135	2972	
top	June	6/2/2019 21:54	6/24/2019 0:30	0:04:00	1898 Mountain Rd NW, Albuquerque, NM 87104, USA	1 NW,

```
df.isnull().sum()
```

	0
month	0
trip_id	0
region_id	0
vehicle_id	0
started_at	0
ended_at	0
DURATION	2308
start_location_name	6
end_location_name	2070
user_id	0
trip_ledger_id	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.dropna(inplace=True)
```

```
df["started_at"] = pd.to_datetime(df["started_at"])  
df["ended_at"] = pd.to_datetime(df["ended_at"])
```

```
print(df.columns.tolist())
```

```
['month', 'trip_id', 'region_id', 'vehicle_id', 'started_at', 'ended_a
```

```
df.dtypes
```

	0
month	object
trip_id	int64
region_id	int64
vehicle_id	int64
started_at	datetime64[ns]
ended_at	datetime64[ns]
DURATION	object
start_location_name	object
end_location_name	object
user_id	int64
trip_ledger_id	int64

dtype: object

df.describe()

	trip_id	region_id	vehicle_id	started_at	ended_at
count	3.003200e+04	30032.0	3.003200e+04	30032	30032
mean	2.005499e+06	202.0	5.584538e+06	2019-06-20 08:54:12.165023744	2019-06-20 09:15:01.030900000
min	1.613335e+06	202.0	1.034847e+06	2019-05-21 18:33:00	2019-05-21 18:33:00
25%	1.796630e+06	202.0	3.253166e+06	2019-06-05 01:27:00	2019-06-05 01:27:00
50%	1.970643e+06	202.0	5.611760e+06	2019-06-17 23:32:30	2019-06-17 23:32:30
75%	2.003200e+06	202.0	8.010071e+06	2019-07-05 01:27:00	2019-07-05 01:27:00

df.describe(include="object")

	month	DURATION	start_location_name	end_location_name
count	30032	30032	30032	30032
unique	3	3927	2792	4137
top	June	0:04:00	1898 Mountain Rd NW, Albuquerque, NM 87104, USA	1898 Mountain Rd NW, Albuquerque, NM 87104, USA


```
df.nunique()
```

	0
month	3
trip_id	30032
region_id	1
vehicle_id	953
started_at	17968
ended_at	17207
DURATION	3927
start_location_name	2792
end_location_name	4137
user_id	11388
trip_ledger_id	30032

dtype: int64

```
print(df["DURATION"].dtype)
```

object

```
print(df["DURATION"].head(10))
```

```
0    0:07:03
1    0:04:57
2    0:01:14
3    0:06:58
4    0:03:06
5    0:21:27
6    0:30:43
7    0:01:05
8    0:05:33
9    0:31:54
```

Name: DURATION, dtype: object

```
df["DURATION"] = pd.to_numeric(df["DURATION"], errors="coerce")
```

```
print(df["DURATION"].dtype)
```

float64

```
import pandas as pd
```

```
df = pd.read_csv("scooter (1).csv")
```

```
df["started_at"] = pd.to_datetime(df["started_at"])
df["ended_at"] = pd.to_datetime(df["ended_at"])

df["DURATION"] = pd.to_timedelta(df["DURATION"])
df["DURATION_MIN"] = df["DURATION"].dt.total_seconds() / 60
```

```
Q1 = df["DURATION_MIN"].quantile(0.25)
Q3 = df["DURATION_MIN"].quantile(0.75)
IQR = Q3 - Q1
```

```
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
```

```
outliers = df[(df["DURATION_MIN"] < lower) | (df["DURATION_MIN"] > upper)]
print(outliers)
```

```

      month  trip_id  region_id  vehicle_id  started_at \
49      May  1638907         202    6754027 2019-05-24 17:35:00
67      May  1639308         202    7730848 2019-05-24 18:01:00
102     May  1640021         202    7261117 2019-05-24 18:49:00
116     May  1640239         202    4942355 2019-05-24 19:01:00
120     May  1640311         202    2672060 2019-05-24 19:04:00
...      ...      ...      ...      ...      ...
34087   July  2479158         202    3312785 2019-07-21 21:55:00
34127   July  2480101         202    5812897 2019-07-21 22:28:00
34220   July  2482224         202    7247079 2019-07-21 23:51:00
34223   July  2482257         202    5136810 2019-07-21 23:52:00
34225   July  2482335         202    9449822 2019-07-21 23:55:00

      ended_at  DURATION \
49  2019-05-24 19:01:00 0 days 01:25:45
67  2019-05-24 19:04:00 0 days 01:03:03
102 2019-05-24 23:55:00 0 days 05:06:04
116 2019-05-24 20:07:00 0 days 01:06:06
120 2019-05-24 20:07:00 0 days 01:02:25
...      ...      ...
34087 2019-07-21 23:03:00 0 days 01:08:00
34127 2019-07-21 23:33:00 0 days 01:05:00
34220 2019-07-22 01:46:00 0 days 01:55:00
34223 2019-07-22 01:45:00 0 days 01:53:00
34225 2019-07-22 01:46:00 0 days 01:51:00

      start_location_name \
49      330 Tijeras Ave NW, Albuquerque, NM 87102, USA
67      401 3rd St NW, Albuquerque, NM 87102, USA
102     Gold Bldg, 320 Gold Ave SW, Albuquerque, NM 87...
116     2550 Central Ave NE, Albuquerque, NM 87106, USA
120     2550 Central Ave NE, Albuquerque, NM 87106, USA
...      ...
34087   1898 Mountain Rd NW, Albuquerque, NM 87104, USA
34127   2000 Mountain Rd NW, Albuquerque, NM 87104, USA
34220   2500 Central Ave SE, Albuquerque, NM 87106, USA
34223    105 Stanford Dr SE, Albuquerque, NM 87106, USA
34225    105 Stanford Dr SE, Albuquerque, NM 87106, USA

      end_location_name  user_id \
49      301 Copper Ave NW, Albuquerque, NM 87102, USA  6813327
```

```

67      330 Tijeras Ave NW, Albuquerque, NM 87102, USA 35526094
102      101 State Hwy 313, Albuquerque, NM 87102, USA 35535567
116      2550 Central Ave NE, Albuquerque, NM 87106, USA 35520946
120      2550 Central Ave NE, Albuquerque, NM 87106, USA 35535261
...
34087 1800 Old Town Rd NW, Albuquerque, NM 87104, USA 42583970
34127      NaN 42586159
34220 3418 Central Ave SE, Albuquerque, NM 87106, USA 42587320
34223 3339 Central Ave NE, Albuquerque, NM 87106, USA 42576631
34225 3339 Central Ave NE, Albuquerque, NM 87106, USA 42586810

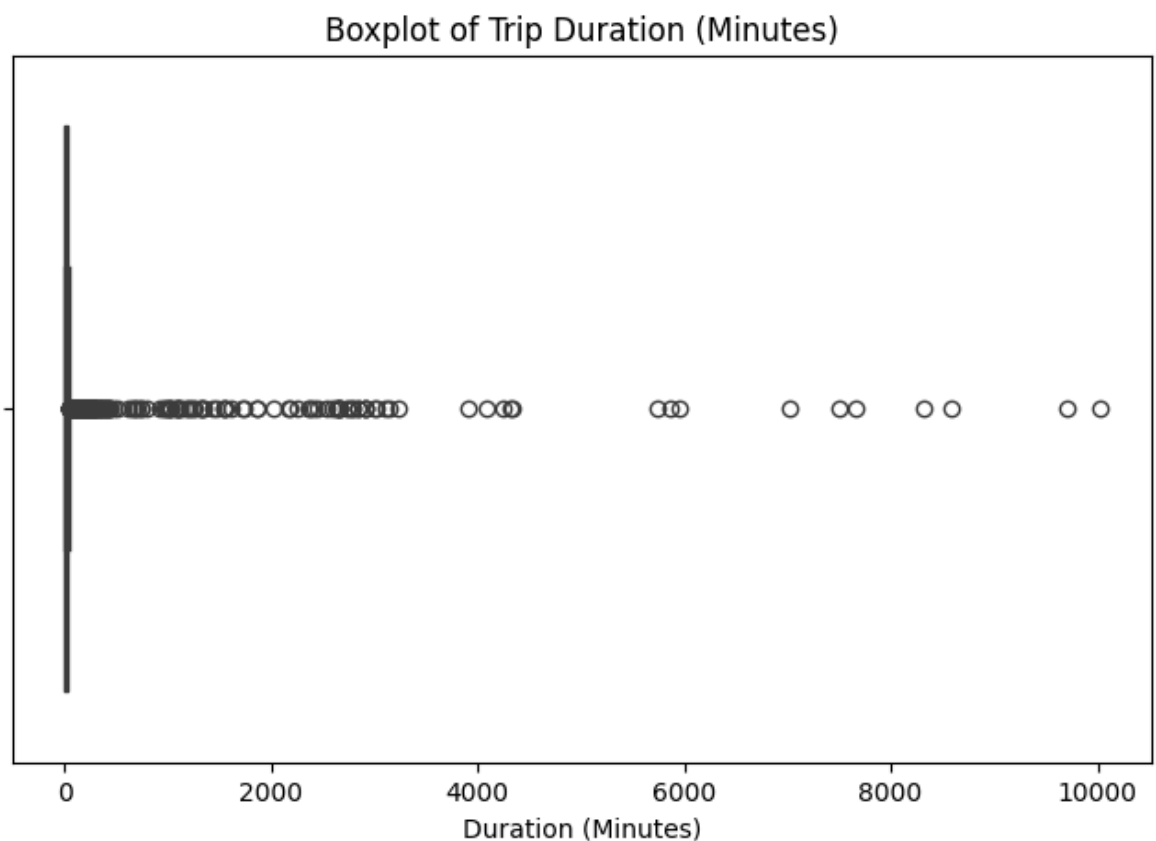
      trip_ledger_id  DURATION_MIN
49          1514427      85.750000
67          1514473      63.050000
102         1519596     306.066667
116         1515560      66.100000
120         1515557      62.416667

```

```

plt.figure(figsize=(8,5))
sns.boxplot(x=df["DURATION_MIN"])
plt.title("Boxplot of Trip Duration (Minutes)")
plt.xlabel("Duration (Minutes)")
plt.show()

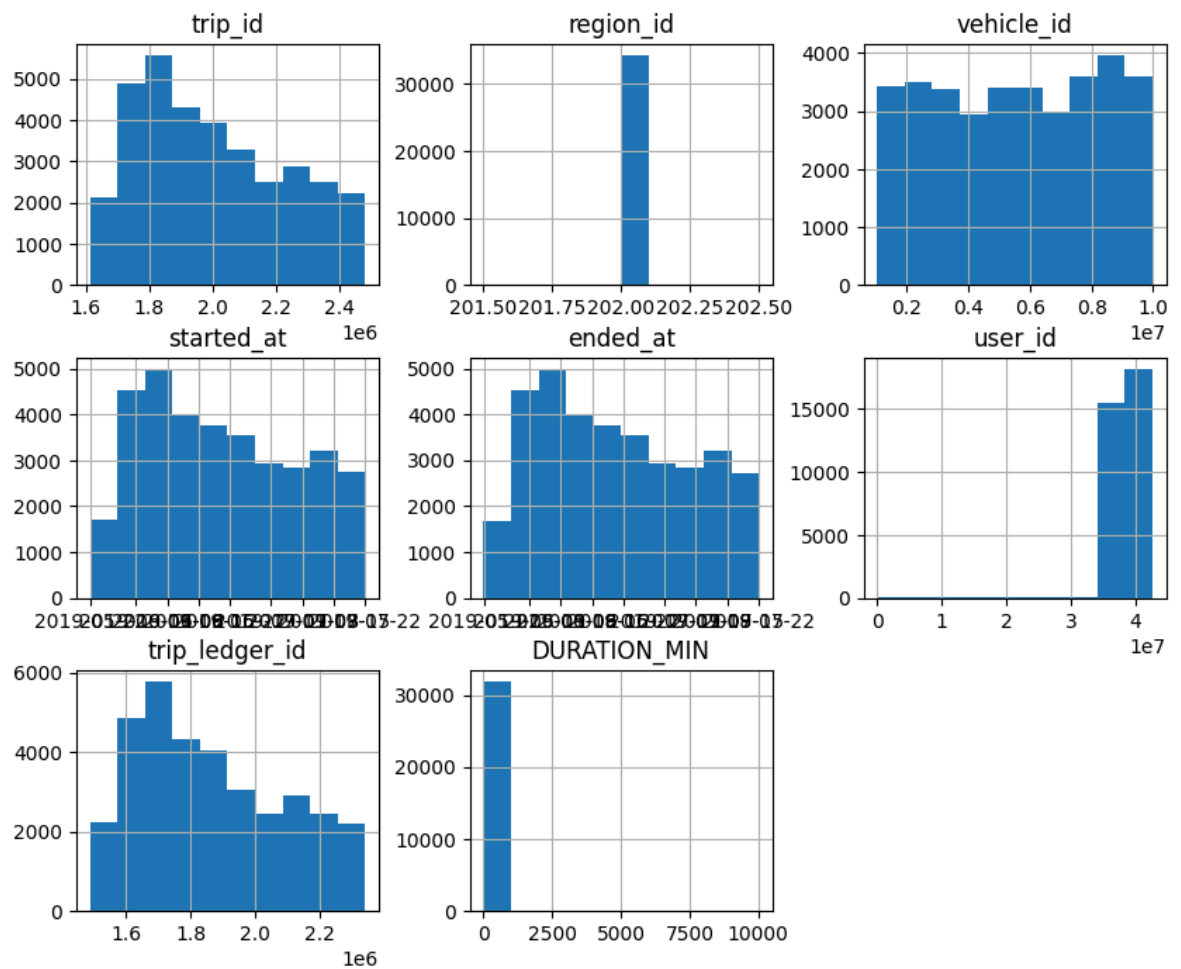
```



```

df.hist(figsize=(10,8))
plt.show()

```



```
sns.heatmap(df.corr(numeric_only=True),
             annot=True,
             cmap="coolwarm")

plt.show()
```

